

03 - Curriculum Integration Guide

Project: Dragon Merchant Guild (*Working Title*)

Document Version: 1.0 (Living Document)

1. Purpose

This document connects the C# curriculum directly to the game's development.

Rather than building random features, every study session unlocks meaningful progress inside the guild.

The curriculum determines **when** mechanics are introduced.

The Game Design Document determines **how** those mechanics behave.

2. Guiding Principle

Every chapter should produce visible progress.

Studying should never feel disconnected from building.

Whenever possible:

Learn → Build → Play → Reflect

3. Progression Structure

Curriculum

↓

New Knowledge

↓

Game Mechanic

↓

Playable Improvement

↓

Reflection

↓

Next Chapter

The player should always finish a study session with something new they can actually interact with.

4. Chapter 1 — C# Fundamentals

Goal

Become comfortable writing small but correct C# programs.

This chapter builds the foundation of the guild.

Value Types vs Reference Types

Learning Goal

Understand copying versus sharing.

Guild Features

- Basic resources
- Dragons
- Workers
- Inventory entries

Suggested Tasks

- Create your first dragon.
- Create your first worker.
- Create several resource types.
- Experiment with copying values versus sharing objects.

Future Refactor

Collections

Nullability & Patterns

Learning Goal

Safely handle missing values.

Guild Features

- Optional contracts
- Missing dragon assignments
- Empty inventory slots
- Optional worker roles

Suggested Tasks

- Find available contracts.
- Search for dragons by name.
- Safely handle missing results.

Future Refactor

LINQ queries.

Methods & the C Corner

Learning Goal

Organize logic into reusable methods.

Guild Features

- Accept Contract
- Feed Dragon
- Hire Worker
- Assign Dragon
- Sell Resources

Suggested Tasks

Implement the guild's first meaningful gameplay actions.

Avoid placing logic inside one giant file.

Errors & Input/Output

Learning Goal

Read input.

Write output.

Handle mistakes gracefully.

Guild Features

- Main menu
- Save reports
- Player input
- Command processing

Suggested Tasks

Create the playable console interface.

Mini Project

Port an older C program into C#.

Suggested candidates:

- Inventory system
 - Menu system
 - Calculator
 - Resource tracker
-

Chapter Checkpoint

By the end of Fundamentals the player should have:

✓ A playable console application

✓ Dragons

✓ Workers

✓ Contracts

✓ Resources

✓ Inventory

✓ Main menu

No advanced architecture is expected yet.

5. Chapter 2 — Object-Oriented Programming

Goal

Transform the project from procedural code into an object-oriented design.

Encapsulation

Introduce proper properties.

Hide implementation details.

Inheritance

Create dragon families.

Worker specializations.

Shared behaviors.

Interfaces

Examples include:

- IWorker
- IDragon
- IContract

Focus on behavior rather than inheritance.

Polymorphism

Allow different dragons to perform work differently while sharing a common interface.

Records

Use records where identity is unimportant.

Examples:

- Resources
 - Rewards
 - Recipes
-

Chapter Checkpoint

The project should feel noticeably cleaner.

Adding new dragon species should require very little duplicated code.

6. Chapter 3 — Collections & Generics

Goal

Replace simple variables with scalable collections.

Expected improvements include:

- Dragon roster
- Worker roster
- Inventory
- Buildings
- Active contracts

Replace repetitive code with loops.

Use dictionaries where appropriate.

Chapter Checkpoint

The guild should now support many dragons and workers naturally.

7. Chapter 4 — LINQ

Goal

Query data instead of manually searching.

Examples include:

Find hungry dragons.

Find contracts with highest reward.

Sort workers.

Search inventory.

Generate reports.

Chapter Checkpoint

The codebase should become significantly shorter and easier to read.

8. Chapter 5 — Async & Await

Goal

Understand asynchronous programming.

Possible future features:

Saving

Loading

Background events

Trade caravans

Research

Long-running guild operations

Chapter Checkpoint

The project should begin feeling like a professional C# application rather than a beginner exercise.

9. Graduation

Completing the curriculum means completing the first era of the guild.

The player should now possess:

- A functioning merchant guild simulation
- Clean C# architecture
- Practical experience
- Confidence reading documentation
- Confidence designing new systems independently

From this point onward, the project enters open development.

New features are no longer driven by the curriculum, but by curiosity, experimentation, and continued learning.

10. Final Reminder

Never skip ahead simply because a feature sounds exciting.

The goal is not to finish the game quickly.

The goal is to become the kind of programmer who could confidently continue expanding it for years.